

Resumen de Clase — Bootcamp JavaScript

Sesión: Prototipos, Herencia, Clases, Gestión de Memoria, NPM, Transpilación y Programación Reactiva (RxJS)

Este documento resume una clase en directo de un bootcamp de JavaScript. La sesión repasa brevemente conceptos de la semana anterior (closures, contexto/*this*, promesas y *async/await*) y luego se centra en la "parte dos": prototipos, herencia, funciones constructoras y clases, gestión de memoria y garbage collector, NPM y versionado semántico, transpilación (Babel) y programación reactiva con RxJS.

0. Repaso rápido de la semana anterior

- **Closures:** funciones que conservan acceso al contexto (variables) en el que fueron creadas; permiten encapsular información no accesible desde fuera. Muy usados en la construcción de frameworks.
- **Promesas:** concepto fundamental que hay que dominar bien; gestionan la asincronía en JavaScript.
- **Async/await:** evolución "azúcar sintáctico" sobre las promesas que simplifica su uso, especialmente al esperar respuestas de *fetch*.
- **Idea clave repetida en toda la clase:** el núcleo ("core") de JavaScript apenas ha cambiado desde sus orígenes. Todo lo nuevo (*async/await*, clases, etc.) son capas de "azúcar sintáctico" que facilitan el desarrollo, pero por debajo el motor sigue funcionando igual (event loop, callbacks, promesas, gestión del contexto).

1. Prototipos y herencia en JavaScript

En JavaScript, absolutamente todo es un objeto (funciones, clases, arrays...). Todos los objetos están interconectados mediante el **prototipo**: la conexión entre un objeto y su "padre". Esta cadena de prototipos llega hasta **Object** (el objeto padre de todos), cuyo propio prototipo es **null**.

- La propiedad `__proto__` de cualquier objeto indica cuál es su padre (útil para inspección en el navegador).
- En código se accede al padre mediante **Object.prototype** / **MiObjeto.prototype**.
- Cuando se llama a un método sobre un objeto, JavaScript busca primero en el propio objeto y, si no lo encuentra, va subiendo por la cadena de prototipos hasta encontrarlo (o hasta llegar a **null**).
- **Regla de buena práctica:** nunca sobrescribir los métodos ya definidos en los objetos predefinidos de JavaScript (**Object**, **Array**, etc.), salvo que sea intencionado y controlado.

Ejemplo trabajado: Usuario y Admin

- Se define una función `Usuario(nombre)` que asigna `this.nombre`.
- El método `saludar` se define como `Usuario.prototype.saludar = function(){...}` — no directamente dentro de la función constructora.
- Se define `Admin(nombre)` que hace `Usuario.call(this, nombre)` para heredar el contexto (mismo mecanismo de `call/apply` visto la semana anterior).

- La herencia se establece con `Admin.prototype = Object.create(Usuario.prototype)`, vinculando Admin como "hijo" de Usuario.

¿Por qué usar `.prototype` y no definir el método dentro de la función?

- Si el método se define dentro de la función constructora, cada instancia creada con "new" se lo lleva copiado en memoria: mayor consumo.
- Si se define en el prototype, el método vive una única vez en el objeto "padre" (la entidad genérica) y todas las instancias acceden a él por herencia: ahorro de memoria.
- Regla general: los datos que cambian por instancia (ej. nombre) van en el propio objeto/this; el comportamiento genérico y reutilizable (ej. métodos) va en el prototype.

Ejemplo Camión / propiedades compartidas

- Si una propiedad (por ejemplo, un array carga) se define en `Camion.prototype.carga = []`, esa propiedad es **compartida por todas las instancias**.
- Consecuencia práctica: si el camión A hace push sobre carga, el camión B "ve" el mismo cambio, porque no es una propiedad de la instancia sino del prototipo (padre) común.
- Esto puede ser intencionado (ej. un recurso compartido tipo "flota") o un error si se esperaba que cada instancia tuviera su propio array independiente.

2. De las funciones constructoras a las clases ("class")

- **new** permite construir instancias de un objeto a partir de una función constructora; es el precursor histórico de las clases.
- **class** (disponible desde ECMAScript 6) es también "azúcar sintáctico": internamente sigue funcionando con prototipos, pero simplifica la sintaxis (constructor, extends, super) y evita tener que gestionar manualmente `Object.create` y `.prototype`.
- JavaScript incorporó `class` porque muchos desarrolladores backend (PHP, Java, etc.) ya estaban familiarizados con esa nomenclatura.
- El **constructor** sustituye a la antigua función constructora y es donde se reciben e inicializan los parámetros con `this`.
- **extends** crea la relación de herencia entre clases; **super** permite invocar al constructor o a los métodos del padre.

Ejemplo: Pedido y PedidoPremium

- `PedidoPremium extends Pedido`: en su constructor recibe `cliente` y `descuento`.
- Regla obligatoria: **lo primero dentro del constructor de una clase hija debe ser la llamada a `super(...)`**, para inicializar correctamente al padre antes de usar `this`.
- El método `total()` de la clase hija sobrescribe al del padre y calcula `super.total() * (1 - descuento)`, es decir, reutiliza la lógica del padre y le aplica el descuento.

Errores típicos detectados en el ejercicio de "Etiquetas / Perfil"

- Error 1: usar "this" antes de llamar a "super()" en el constructor de la clase hija — siempre debe ir "super" primero.
- Error 2: asignar una referencia a un método (ej. `const guardar = perfil.añadir`) sin conservar el contexto ("this") de la instancia original. La solución es usar `".bind(perfil)"` para fijar el contexto, igual que se vio la semana anterior con `call/apply/bind`.

3. Gestión de memoria y Garbage Collector

JavaScript ejecuta periódicamente un proceso de limpieza (garbage collector) que libera la memoria de todo aquello que ya no tiene ninguna referencia activa. Toda la memoria usada por una pestaña del navegador se traspa al ordenador del usuario (por eso Chrome puede consumir tanta memoria con muchas pestañas abiertas).

Principales causas de fugas de memoria ("memory leaks") mencionadas:

- **Variables globales:** siempre tienen una referencia activa (nunca se liberan). Recomendación: evitar "var" y variables globales; usar siempre `const/let` con el contexto adecuado.
- **setInterval / timers activos:** si no se detienen, siguen ejecutándose indefinidamente aunque el componente que los creó ya no exista (ej. al navegar a otra página). Hay que guardar el ID que devuelve `setInterval` y llamar a `clearInterval(id)` al destruir el componente.
- **Event listeners "fantasma":** un listener puede quedar escuchando eventos aunque el elemento asociado ya no exista. Es imprescindible guardar una referencia a la función y usar `removeEventListener(evento, funcionReferenciada)` al desmontar.
- **Closures mal gestionados:** pueden retener memoria innecesariamente si no se liberan sus referencias.

Este patrón de "crear al montar / limpiar al desmontar" es el mismo principio que luego se aplica en el ciclo de vida de los Web Components y, más adelante, en Angular.

4. NPM y gestión de dependencias

- **NPM** (Node Package Manager) es el gestor de paquetes de Node.js, usado de forma genérica en cualquier proyecto JavaScript (Angular, React, Vue, etc.), no exclusivo de ningún framework.
- Alternativa equivalente: **Yarn**. Ambos cumplen la misma función; según el caso, uno puede ser más rápido o conveniente que el otro.
- `npm init` genera el archivo **package.json**, que documenta el proyecto (nombre, versión, autor, descripción, dependencias, etc.).
- `npm install` descarga e instala las dependencias declaradas en `package.json`.
- **package-lock.json** bloquea las versiones exactas instaladas para garantizar que producción use lo mismo que se probó en desarrollo. No se edita manualmente, se genera solo.

- **dependencies** vs **devDependencies**: las primeras son necesarias para que la aplicación funcione en producción; las segundas solo se usan durante el desarrollo (herramientas auxiliares) y no son necesarias en producción.

Versionado semántico (SemVer): Mayor.Minor.Patch

- **Mayor (major)**: cambios grandes, puede romper la retrocompatibilidad. Actualizar con cuidado.
- **Minor**: nueva funcionalidad añadida manteniendo la retrocompatibilidad. En teoría, sin riesgo.
- **Patch**: corrección rápida de un error (incluso crítico) en producción. Se recomienda actualizar de inmediato.
- Notación en package.json: fijar versión exacta (ej. 2.4.7) instala esa versión concreta; con **virgulilla (~)** se permiten actualizaciones de patch; con **caret (^)** se permiten actualizaciones de minor y patch.

5. Transpilación (Babel)

La transpilación permite convertir código escrito en una versión moderna de JavaScript (ej. ECMAScript 6) a una versión anterior compatible con más entornos/navegadores, y también integrar librerías o código antiguo para que funcione junto con código nuevo. La herramienta de referencia mencionada es **Babel**. Este proceso ocurre en el momento de la construcción (build) de la aplicación.

6. Programación reactiva: RxJS

La programación reactiva es un salto respecto a las promesas: mientras que una promesa se resuelve una única vez (éxito o error) y se cierra, en programación reactiva existe un **flujo continuo y abierto de datos** a lo largo del tiempo.

Conceptos clave

- **Observable**: la fuente que "emite" datos a lo largo del tiempo (puede crearse a partir de eventos, intervalos, promesas, arrays, etc., mediante funciones de la librería como `fromEvent`, `of...`). Por convención, las variables que representan observables se nombran terminadas en `$`.
- **Pipe ("tubería")**: permite encadenar **operadores** (`map`, `filter`, `debounceTime`, `distinctUntilChanged`, `reduce`, `take`, etc.) que transforman o filtran los datos a medida que pasan por el flujo.
- **Suscripción (subscribe)**: es obligatoria para poder recibir los datos — un observable sin suscripción no emite nada visible (analogía usada: sin suscripción a Netflix no se puede ver el contenido). El objeto de suscripción define tres callbacks: **next** (cada valor recibido), **error** (si ocurre un fallo) y **complete** (cuando el flujo se cierra).
- Es imprescindible cancelar la suscripción (`unsubscribe`) cuando ya no se necesita, igual que con intervals y event listeners, para evitar fugas de memoria.

Ejemplo resuelto: `of(1,2,3,4).pipe(map, filter, reduce)`

- `map(x => x * 2)`: transforma la serie `[1,2,3,4]` en `[2,4,6,8]`.
- `filter(x => x > 4)`: se queda solo con `[6, 8]`.

- `reduce((acc, val) => acc + val, 0)`: suma los valores partiendo de un valor inicial 0, dando como resultado **14**.
- El valor inicial (0) en `reduce` es el punto de partida de la acumulación — igual que en el `reduce` de un array normal.

Unicast vs. Multicast (Subject)

- **Unicast (Observable normal)**: cada suscriptor recibe su propia ejecución independiente; si alguien se suscribe más tarde, empieza a recibir datos desde ese momento, y los valores pueden ser distintos para cada suscriptor.
- **Multicast (Subject)**: todos los suscriptores reciben exactamente el mismo evento/mensaje al mismo tiempo, compartiendo la misma fuente de emisión.
- Se mencionan brevemente los **Schedulers** como un tema más avanzado, a explorar más adelante por cuenta propia.

Cierre de la sesión

El formador cierra la clase indicando que la siguiente semana no habrá sesión de este módulo porque el grupo tiene metodologías ágiles, y que retomarán en torno al día 30–31 con el tema de maquetación.

Documento generado a partir de la transcripción de la grabación de la clase ("Su grabación 5.wav"). Resume las ideas y explicaciones dadas por el formador; se recomienda contrastar con el material oficial del curso.